

1. PRE-TEST (Waktu: 20 Menit)

Buat program kalkulator pembagian sederhana menggunakan Java OOP. Program ini akan meminta pengguna untuk memasukkan dua buah angka melalui keyboard: angka pertama sebagai pembilang dan angka kedua sebagai penyebut. Mengingat input dari pengguna seringkali tidak terduga, Anda harus menerapkan penanganan error menggunakan blok try-catch-finally. Tangani secara spesifik kasus di mana pengguna memasukkan angka nol sebagai penyebut, yang akan memicu `ArithmeticException`, serta kasus di mana pengguna memasukkan huruf atau karakter non-numerik lainnya, yang akan memicu `InputMismatchException`. Pastikan Anda menampilkan pesan peringatan yang informatif dan mudah dimengerti untuk setiap jenis kesalahan tersebut. Terakhir, gunakan blok finally untuk menutup objek `Scanner` yang digunakan untuk mengambil input, dengan mencetak pesan bahwa "Proses kalkulasi selesai dan resource memori telah dibersihkan."

2. LATIHAN 1

Soal 1: Persiapan Sistem dan Kuota Kelas

Sebagai langkah awal, admin akademik perlu memasukkan data kuota untuk 3 mata kuliah utama ke dalam sistem. Buatlah program yang menggunakan array integer berukuran 3 untuk menyimpan data kuota tersebut. Gunakan perulangan untuk meminta admin menginputkan kuota satu per satu. Anda harus mengantisipasi dua kesalahan: pertama, cegah program *crash* jika admin memasukkan huruf (bukan angka) dengan menangkap `InputMismatchException`. Kedua, buat simulasi di mana perulangan sengaja berjalan sebanyak 4 kali untuk memicu kesalahan batas array, lalu tangkap kesalahan tersebut menggunakan `ArrayIndexOutOfBoundsException`. Tampilkan pesan error yang sesuai untuk setiap kondisi tanpa menghentikan program.

Soal 2: Validasi Data Mahasiswa

Langkah selanjutnya adalah memvalidasi data mahasiswa yang akan mengisi KRS. Buatlah kelas `Mahasiswa` yang memiliki metode `setSksMaksimal(int sks)`. Metode ini bertugas menentukan batas maksimal SKS yang bisa diambil mahasiswa berdasarkan IPK mereka. Namun, sistem memiliki aturan baku: tidak ada mahasiswa yang boleh mengambil lebih dari 24 SKS atau kurang dari 2 SKS. Jika argumen SKS yang dimasukkan melanggar aturan ini, gunakan keyword `throw` secara eksplisit untuk melempar `IllegalArgumentException` dengan pesan: "Kesalahan sistem: Batas SKS tidak valid (harus antara 2 - 24 SKS)!". Uji metode ini di dalam blok try-catch pada kelas utama.

Soal 3: Penambahan Mata Kuliah

Mahasiswa kini mulai memilih mata kuliah. Buatlah sebuah *Custom Exception* bertipe *Unchecked* (mewarisi `RuntimeException`) bernama `SksTidakCukupException`. Pada kelas `Mahasiswa` (lanjutan dari Soal 2), tambahkan atribut `sisaSks` dan metode `ambilMataKuliah(String namaMatkul, int bebanSks)`. Di dalam metode ini, buat logika pengecekan: jika `bebanSks` dari mata kuliah yang dipilih ternyata lebih besar dari `sisaSks` yang dimiliki mahasiswa, lemparkan objek `SksTidakCukupException` yang telah Anda buat. Pada kelas `Main`, simulasikan seorang mahasiswa yang `sisa SKS`-nya tidak cukup mencoba mengambil mata kuliah yang berat, lalu tangkap errornya.

Soal 4: Pengecekan Ketersediaan Kelas

Selain mengecek SKS mahasiswa, sistem juga harus mengecek kapasitas kelas. Buatlah *Custom Exception* bertipe *Checked* (mewarisi kelas `Exception`) dengan nama `KelasPenuhException`. Tambahkan

kelas baru bernama SistemAkademik yang memiliki metode gabungKelas(String kodeKelas, int kuotaTersedia). Karena Anda menggunakan *checked exception*, metode ini wajib menambahkan deklarasi throws KelasPenuhException pada *signature*-nya. Simulasikan logika: jika kuotaTersedia bernilai 0 atau kurang, lempar exception tersebut. Panggil metode ini dari kelas utama dengan kuota 0, dan penuhi kewajiban *compiler* dengan membungkus pemanggilannya di dalam blok try-catch.

Soal 5: Pencetakan Dokumen KRS

Tahap terakhir, mahasiswa mencetak dokumen KRS mereka. Tambahkan metode cetakDokumenKrs(String namaFile) pada kelas SistemAkademik yang mendeklarasikan throws FileNotFoundException (impor dari java.io.FileNotFoundException). Jika parameter namaFile yang dimasukkan bukan "krs_valid.txt", maka lemparkan FileNotFoundException secara manual menggunakan throw new. Di kelas utama, eksekusi metode pencetakan ini di dalam blok try-catch. Tangkap dan cetak pesan errornya jika nama file salah. Untuk mengakhiri seluruh proses KRS, gunakan blok finally yang wajib mencetak pesan "Sesi Sistem Rencana Studi telah ditutup. Koneksi database diputuskan," terlepas dari berhasil atau gagalnya pencetakan dokumen.

2. LATIHAN 2

Soal 1: Input Harga Menu Dasar

Sebagai kasir, Anda perlu memasukkan harga untuk 3 menu utama hari ini ke dalam sistem. Buatlah program yang menggunakan array integer berukuran 3 untuk menyimpan harga tersebut. Gunakan perulangan for untuk meminta kasir menginputkan harga satu per satu melalui keyboard (Scanner). Tugas Anda:

1. Sengaja buat perulangan berjalan sebanyak 4 kali untuk memicu error batas array.
2. Gunakan blok try-catch untuk menangkap dua kemungkinan error:
 - InputMismatchException: Tangkap jika kasir keliru mengetikkan huruf (misal: "lima ribu") alih-alih angka. Tampilkan pesan: "Error: Input harga harus berupa angka!".
 - ArrayIndexOutOfBoundsException: Tangkap jika sistem mencoba memasukkan harga ke-4 pada array yang hanya berukuran 3. Tampilkan pesan: "Error: Kapasitas memori harga sudah penuh!".

Soal 2: Validasi Umur Pelanggan

Cafe ini memiliki aturan bahwa hanya pelanggan berusia 17 tahun ke atas yang boleh mendaftar sebagai Member VIP. Buatlah kelas Pelanggan yang memiliki metode daftarMember(int umur). Tugas Anda: Di dalam metode tersebut, buat pengecekan if. Jika umur kurang dari 17, gunakan keyword throw untuk melempar IllegalArgumentException dengan pesan: "Maaf, umur Anda belum mencukupi untuk menjadi Member VIP". Uji metode ini di kelas utama di dalam blok try-catch dengan memasukkan umur 15, lalu cetak pesan errornya.

Soal 3: Cek Ketersediaan Kopi

Kadang-kadang stok biji kopi bisa habis. Buatlah sebuah *Custom Exception* bertipe *Unchecked* (artinya: mewarisi kelas *RuntimeException*) yang diberi nama *KopiHabisException*. Tugas Anda: Pada kelas *Pelanggan* (lanjutan Soal 2), tambahkan atribut *stokKopi* (beri nilai awal 5) dan metode *pesanKopi*(int *jumlahPesanan*). Jika *jumlahPesanan* lebih besar dari *stokKopi*, lemparkan *KopiHabisException* yang sudah Anda buat. Jika cukup, kurangi stoknya. Simulasikan di kelas utama dengan memesan kopi sebanyak 10 gelas, dan tangkap errornya.

Soal 4: Pembayaran di Kasir

Saatnya pelanggan membayar. Buatlah *Custom Exception* bertipe *Checked* (artinya: mewarisi kelas *Exception*) bernama *UangKurangException*. Tugas Anda: Buat kelas baru bernama *MesinKasir* dengan metode *bayar*(int *totalBelanja*, int *uangDiberikan*). Karena ini *checked exception*, metode ini **wajib** menggunakan keyword *throws* *UangKurangException* di sebelah nama metodenya. Jika *uangDiberikan* lebih kecil dari *totalBelanja*, lempar exception tersebut. Uji metode ini di kelas utama (misal: total belanja Rp 50.000, tapi uang yang diberikan hanya Rp 30.000) dan bungkus dengan *try-catch*.

Soal 5: Mencetak Struk

Langkah terakhir adalah mencetak struk belanja. Pada kelas *MesinKasir*, tambahkan metode *cetakStruk*(boolean *statusPrinter*) yang mendeklarasikan *throws* *Exception*. Tugas Anda: Jika *statusPrinter* bernilai *false* (kertas struk habis), lemparkan *Exception* bawaan Java dengan pesan "Printer error: Kertas struk habis!". Panggil metode ini di kelas utama dengan nilai parameter *false* di dalam blok *try-catch*. Terakhir, tambahkan blok *finally* yang **selalu** mencetak pesan penutup: "Terima kasih telah berkunjung ke Cafe Java Bean. Program kasir ditutup." untuk menyimulasikan akhir dari program.